ĐỀ TÀI 01. Frozen Lake nhiều mức trơn

Mục tiêu
Agent đi từ Start đến Goal trên lưới có hố. Điểm chính là mô hình hóa chuyển trạng thái ngẫu nhiên do mặt băng trơn.

Mô hình MDP
State: (hàng, cột); nếu có cơ chế đặc biệt thì bổ sung trạng thái tương ứng.
Action: Lên, xuống, trái, phải.
Reward gợi ý: Thưởng khi tới Goal; phạt khi rơi hố; có thể phạt nhẹ theo bước để khuyến khích đường ngắn.

Các setting bắt buộc
1. Map 5×5 deterministic.
2. Map 8×8, mỗi action có xác suất trượt sang hướng bên cạnh.
3. Map 10×10 có nhiều vùng băng với xác suất trượt khác nhau.
4. Thêm ô 'giày chống trượt': sau khi ăn, 3 bước tiếp theo action trở thành deterministic.

Yêu cầu riêng
So sánh policy khi thay đổi xác suất trượt; giải thích vì sao đường ngắn nhất hình học chưa chắc là đường tối ưu.

Môi trường / thư viện gợi ý

Gymnasium `FrozenLake-v1` làm baseline; dùng `desc`/`is_slippery` cho map cơ bản, sau đó subclass `gymnasium.Env` để thêm vùng trơn khác nhau và vật phẩm. Thư viện: gymnasium, numpy, matplotlib; pygame chỉ để render.

Thuật toán
Value Iteration và Q-learning; khuyến khích thêm SARSA. Không sử dụng DQN hoặc function approximation.

Yêu cầu thực nghiệm
• Xây dựng ít nhất 3 map/configuration có độ khó tăng dần; báo cáo kích thước state space và action space.
• Trình bày rõ transition, terminal condition và các biến bổ sung để đảm bảo tính Markov.
• Chạy nhiều seed/episode phù hợp, vẽ đường học hoặc thống kê return; trực quan hóa policy cuối cùng trên ít nhất 2 setting.
• So sánh ít nhất 2 thuật toán hoặc 2 biến thể chính sách theo yêu cầu của đề tài; phân tích trường hợp thuật toán thất bại/chậm hội tụ.
• Thử ít nhất 2 cấu hình reward hoặc tham số môi trường và giải thích policy thay đổi.

Sản phẩm tối thiểu
Mã nguồn environment + agent; file cấu hình/map; báo cáo mô tả MDP, thuật toán, thí nghiệm, kết quả; chương trình demo một episode bằng policy đã học.

ĐỀ TÀI 02. Cliff Walking với vùng an toàn

Mục tiêu
Agent đi từ Start tới Goal, cạnh bản đồ có vực gây phạt lớn và đưa agent về Start.

Mô hình MDP
State: (hàng, cột).
Action: Lên, xuống, trái, phải.
Reward gợi ý: -1 mỗi bước; phạt lớn khi rơi vực; thưởng nhỏ khi tới Goal.

Các setting bắt buộc
1. Map chuẩn nhỏ.
2. Map dài hơn với nhiều đoạn cliff.
3. Thêm ô an toàn: đứng trên đó một bước sẽ tạo checkpoint.
4. Checkpoint chỉ được dùng 1 lần để quay lại vị trí an toàn sau khi rơi vực.

Yêu cầu riêng
Bắt buộc phân tích khác biệt giữa SARSA và Q-learning về mức độ 'mạo hiểm' của policy.

Môi trường / thư viện gợi ý

Gymnasium `CliffWalking-v1` làm baseline; sinh viên sửa map/reward/checkpoint trong custom `gymnasium.Env`. Thư viện: gymnasium, numpy, matplotlib.

Thuật toán
SARSA và Q-learning. Không sử dụng DQN hoặc function approximation.

Yêu cầu thực nghiệm
• Xây dựng ít nhất 3 map/configuration có độ khó tăng dần; báo cáo kích thước state space và action space.
• Trình bày rõ transition, terminal condition và các biến bổ sung để đảm bảo tính Markov.
• Chạy nhiều seed/episode phù hợp, vẽ đường học hoặc thống kê return; trực quan hóa policy cuối cùng trên ít nhất 2 setting.
• So sánh ít nhất 2 thuật toán hoặc 2 biến thể chính sách theo yêu cầu của đề tài; phân tích trường hợp thuật toán thất bại/chậm hội tụ.
• Thử ít nhất 2 cấu hình reward hoặc tham số môi trường và giải thích policy thay đổi.

Sản phẩm tối thiểu
Mã nguồn environment + agent; file cấu hình/map; báo cáo mô tả MDP, thuật toán, thí nghiệm, kết quả; chương trình demo một episode bằng policy đã học.

ĐỀ TÀI 03. Windy Grid World

Mục tiêu
Một số cột có gió đẩy agent thêm một số ô sau khi thực hiện action.

Mô hình MDP
State: (hàng, cột).
Action: Lên, xuống, trái, phải.
Reward gợi ý: -1 mỗi bước, thưởng khi đến Goal.

Các setting bắt buộc
1. Gió cố định theo cột.
2. Map lớn hơn, cường độ gió 0–2 ô.
3. Gió ngẫu nhiên: cường độ thực tế lệch ±1 so với mức danh định.
4. Thêm ô 'neo': nhặt được cho phép bỏ qua tác dụng của gió đúng 1 lần.

Yêu cầu riêng
State phải biểu diễn việc agent còn hay đã dùng quyền bỏ qua gió.

Môi trường / thư viện gợi ý

Custom `gymnasium.Env` dựa trên mẫu GridWorld chính thức; có thể tham khảo Windy Gridworld trong Sutton & Barto. Thư viện: gymnasium, numpy, matplotlib; pygame cho render.

Thuật toán
SARSA và Q-learning; có thể đối chiếu Value Iteration ở setting nhỏ. Không sử dụng DQN hoặc function approximation.

Yêu cầu thực nghiệm
• Xây dựng ít nhất 3 map/configuration có độ khó tăng dần; báo cáo kích thước state space và action space.
• Trình bày rõ transition, terminal condition và các biến bổ sung để đảm bảo tính Markov.
• Chạy nhiều seed/episode phù hợp, vẽ đường học hoặc thống kê return; trực quan hóa policy cuối cùng trên ít nhất 2 setting.
• So sánh ít nhất 2 thuật toán hoặc 2 biến thể chính sách theo yêu cầu của đề tài; phân tích trường hợp thuật toán thất bại/chậm hội tụ.
• Thử ít nhất 2 cấu hình reward hoặc tham số môi trường và giải thích policy thay đổi.

Sản phẩm tối thiểu
Mã nguồn environment + agent; file cấu hình/map; báo cáo mô tả MDP, thuật toán, thí nghiệm, kết quả; chương trình demo một episode bằng policy đã học.

ĐỀ TÀI 04. Key–Door Grid World

Mục tiêu
Agent phải lấy chìa khóa trước khi đi qua cửa để tới Goal.

Mô hình MDP
State: (hàng, cột, has_key).
Action: Lên, xuống, trái, phải.
Reward gợi ý: Thưởng khi lấy key và tới Goal; phạt theo bước; đi vào cửa khi chưa có key không thành công.

Các setting bắt buộc
1. 1 key, 1 door.
2. Map lớn hơn có nhiều đường vòng.
3. Key chỉ dùng được 1 lần và biến mất sau khi mở cửa.
4. Thêm ô thưởng cho phép tạo lại key một lần nếu đã dùng.

Yêu cầu riêng
Sinh viên phải chỉ ra tại sao chỉ dùng vị trí làm state sẽ làm mất tính Markov.

Môi trường / thư viện gợi ý

MiniGrid `MiniGrid-DoorKey-*` dùng làm môi trường tham khảo/khởi đầu; với bài tabular phải chuyển observation thành state rời rạc đầy đủ như (row,col,has_key). Thư viện: minigrid, gymnasium, numpy.

Thuật toán
Value Iteration hoặc Policy Iteration và Q-learning. Không sử dụng DQN hoặc function approximation.

Yêu cầu thực nghiệm
• Xây dựng ít nhất 3 map/configuration có độ khó tăng dần; báo cáo kích thước state space và action space.
• Trình bày rõ transition, terminal condition và các biến bổ sung để đảm bảo tính Markov.
• Chạy nhiều seed/episode phù hợp, vẽ đường học hoặc thống kê return; trực quan hóa policy cuối cùng trên ít nhất 2 setting.
• So sánh ít nhất 2 thuật toán hoặc 2 biến thể chính sách theo yêu cầu của đề tài; phân tích trường hợp thuật toán thất bại/chậm hội tụ.
• Thử ít nhất 2 cấu hình reward hoặc tham số môi trường và giải thích policy thay đổi.

Sản phẩm tối thiểu
Mã nguồn environment + agent; file cấu hình/map; báo cáo mô tả MDP, thuật toán, thí nghiệm, kết quả; chương trình demo một episode bằng policy đã học.

ĐỀ TÀI 05. Colored Keys & Doors

Mục tiêu
Có nhiều loại chìa khóa và cửa tương ứng; agent phải quyết định thứ tự thu thập.

Mô hình MDP
State: (hàng, cột, tập key đang giữ hoặc bitmask).
Action: Lên, xuống, trái, phải.
Reward gợi ý: Thưởng khi mở đúng cửa và hoàn thành; phạt nhẹ theo bước; mở sai cửa không được.

Các setting bắt buộc
1. 2 màu key/door.
2. 3 màu, map lớn hơn.
3. Mỗi key chỉ mở được 1 cửa.
4. Có ô 'master key' dùng đúng 1 lần cho bất kỳ cửa nào.

Yêu cầu riêng
Yêu cầu thiết kế encoding state gọn; so sánh kích thước state space giữa các setting.

Môi trường / thư viện gợi ý

MiniGrid DoorKey/LockedRoom làm khung đối tượng Key–Door; khuyến nghị tự mã hóa state bằng bitmask thay vì dùng ảnh quan sát. Thư viện: minigrid, gymnasium, numpy.

Thuật toán
Policy Iteration trên setting nhỏ và Q-learning trên tất cả setting. Không sử dụng DQN hoặc function approximation.

Yêu cầu thực nghiệm
• Xây dựng ít nhất 3 map/configuration có độ khó tăng dần; báo cáo kích thước state space và action space.
• Trình bày rõ transition, terminal condition và các biến bổ sung để đảm bảo tính Markov.
• Chạy nhiều seed/episode phù hợp, vẽ đường học hoặc thống kê return; trực quan hóa policy cuối cùng trên ít nhất 2 setting.
• So sánh ít nhất 2 thuật toán hoặc 2 biến thể chính sách theo yêu cầu của đề tài; phân tích trường hợp thuật toán thất bại/chậm hội tụ.
• Thử ít nhất 2 cấu hình reward hoặc tham số môi trường và giải thích policy thay đổi.

Sản phẩm tối thiểu
Mã nguồn environment + agent; file cấu hình/map; báo cáo mô tả MDP, thuật toán, thí nghiệm, kết quả; chương trình demo một episode bằng policy đã học.

ĐỀ TÀI 06. Teleport Grid với portal hữu hạn

Mục tiêu
Đi vào portal sẽ dịch chuyển agent tới vị trí ghép cặp.

Mô hình MDP
State: (hàng, cột, số lần portal đặc biệt còn dùng).
Action: Lên, xuống, trái, phải.
Reward gợi ý: Thưởng Goal; phạt bước; portal xấu có thể gây penalty.

Các setting bắt buộc
1. 1 cặp portal cố định.
2. Nhiều cặp portal, có portal một chiều.
3. Một portal tắt sau lần dùng đầu tiên.
4. Ô năng lượng có thể nạp lại đúng 1 lần sử dụng portal đã tắt.

Yêu cầu riêng
Bắt buộc có ít nhất một map mà dùng portal không phải lúc nào cũng tốt.

Môi trường / thư viện gợi ý

Custom `gymnasium.Env` từ mẫu GridWorld; portal được cài trực tiếp trong hàm transition.
Thư viện: gymnasium, numpy, matplotlib; pygame tùy chọn.

Thuật toán
Value Iteration và Q-learning. Không sử dụng DQN hoặc function approximation.

Yêu cầu thực nghiệm
• Xây dựng ít nhất 3 map/configuration có độ khó tăng dần; báo cáo kích thước state space và action space.
• Trình bày rõ transition, terminal condition và các biến bổ sung để đảm bảo tính Markov.
• Chạy nhiều seed/episode phù hợp, vẽ đường học hoặc thống kê return; trực quan hóa policy cuối cùng trên ít nhất 2 setting.
• So sánh ít nhất 2 thuật toán hoặc 2 biến thể chính sách theo yêu cầu của đề tài; phân tích trường hợp thuật toán thất bại/chậm hội tụ.
• Thử ít nhất 2 cấu hình reward hoặc tham số môi trường và giải thích policy thay đổi.

Sản phẩm tối thiểu
Mã nguồn environment + agent; file cấu hình/map; báo cáo mô tả MDP, thuật toán, thí nghiệm, kết quả; chương trình demo một episode bằng policy đã học.

ĐỀ TÀI 07. Treasure Hunt với sức chứa hữu hạn

Mục tiêu
Agent thu thập kho báu trước khi tới Exit; túi có sức chứa giới hạn.

Mô hình MDP
State: (hàng, cột, treasure mask, số món đang mang).
Action: Lên, xuống, trái, phải; có thể thêm Drop nếu chọn setting nâng cao.
Reward gợi ý: Mỗi treasure có reward; Exit chỉ cho thưởng lớn khi đạt điều kiện.

Các setting bắt buộc
1. 2 treasure, không giới hạn túi.
2. 3–4 treasure, túi chỉ chứa 2 món.
3. Có treasure nặng chiếm 2 đơn vị sức chứa.
4. Có ô 'kho chứa' cho phép gửi đồ rồi quay lại lấy.

Yêu cầu riêng
Phải phân tích trade-off giữa đường đi ngắn và giá trị treasure.

Môi trường / thư viện gợi ý

Custom `gymnasium.Env`; map lưu bằng mảng NumPy hoặc ASCII map, treasure dùng bitmask. Thư viện: gymnasium, numpy, matplotlib.

Thuật toán
Monte Carlo và Q-learning. Không sử dụng DQN hoặc function approximation.

Yêu cầu thực nghiệm
• Xây dựng ít nhất 3 map/configuration có độ khó tăng dần; báo cáo kích thước state space và action space.
• Trình bày rõ transition, terminal condition và các biến bổ sung để đảm bảo tính Markov.
• Chạy nhiều seed/episode phù hợp, vẽ đường học hoặc thống kê return; trực quan hóa policy cuối cùng trên ít nhất 2 setting.
• So sánh ít nhất 2 thuật toán hoặc 2 biến thể chính sách theo yêu cầu của đề tài; phân tích trường hợp thuật toán thất bại/chậm hội tụ.
• Thử ít nhất 2 cấu hình reward hoặc tham số môi trường và giải thích policy thay đổi.

Sản phẩm tối thiểu
Mã nguồn environment + agent; file cấu hình/map; báo cáo mô tả MDP, thuật toán, thí nghiệm, kết quả; chương trình demo một episode bằng policy đã học.

ĐỀ TÀI 08. Ordered Treasure Hunt

Mục tiêu
Các vật phẩm phải được thu thập theo một thứ tự hợp lệ trước khi thoát.

Mô hình MDP
State: (hàng, cột, chỉ số nhiệm vụ hiện tại).
Action: Lên, xuống, trái, phải.
Reward gợi ý: Thưởng khi lấy đúng vật tiếp theo; phạt nếu chạm vật sai thứ tự; thưởng lớn khi hoàn thành.

Các setting bắt buộc
1. Chuỗi A→B→C.
2. Có hai chuỗi hợp lệ khác nhau.
3. Một vật có thể bị khóa cho tới khi bật switch.
4. Có ô thưởng cho phép bỏ qua đúng 1 phần tử trong chuỗi.

Yêu cầu riêng
Phải chứng minh bằng ví dụ rằng cùng một vị trí nhưng tiến độ nhiệm vụ khác nhau là các state khác nhau.

Môi trường / thư viện gợi ý

Custom `gymnasium.Env`; có thể tái sử dụng khung Treasure Hunt, thêm `task_progress` vào state. Thư viện: gymnasium, numpy, matplotlib.

Thuật toán
Policy Iteration và Q-learning. Không sử dụng DQN hoặc function approximation.

Yêu cầu thực nghiệm
• Xây dựng ít nhất 3 map/configuration có độ khó tăng dần; báo cáo kích thước state space và action space.
• Trình bày rõ transition, terminal condition và các biến bổ sung để đảm bảo tính Markov.
• Chạy nhiều seed/episode phù hợp, vẽ đường học hoặc thống kê return; trực quan hóa policy cuối cùng trên ít nhất 2 setting.
• So sánh ít nhất 2 thuật toán hoặc 2 biến thể chính sách theo yêu cầu của đề tài; phân tích trường hợp thuật toán thất bại/chậm hội tụ.
• Thử ít nhất 2 cấu hình reward hoặc tham số môi trường và giải thích policy thay đổi.

Sản phẩm tối thiểu
Mã nguồn environment + agent; file cấu hình/map; báo cáo mô tả MDP, thuật toán, thí nghiệm, kết quả; chương trình demo một episode bằng policy đã học.

ĐỀ TÀI 09. Coin Collector với coin biến mất

Mục tiêu
Agent tối đa hóa tổng coin thu được trước khi tới Exit hoặc hết số bước.

Mô hình MDP
State: (hàng, cột, coin mask, thời gian hoặc phase nếu cần).
Action: Lên, xuống, trái, phải.
Reward gợi ý: Reward theo giá trị coin; phạt nhẹ mỗi bước.

Các setting bắt buộc
1. Coin cố định.
2. Coin có giá trị khác nhau.
3. Một số coin biến mất sau N bước.
4. Ô 'time bonus' cộng thêm 3 bước nhưng chỉ dùng một lần.

Yêu cầu riêng
Phải thử ít nhất 2 cách reward và giải thích policy thay đổi.

Môi trường / thư viện gợi ý

Custom `gymnasium.Env`; map ASCII/NumPy, coin mask và time limit. Thư viện: gymnasium, numpy, matplotlib.

Thuật toán
Monte Carlo và Q-learning. Không sử dụng DQN hoặc function approximation.

Yêu cầu thực nghiệm
• Xây dựng ít nhất 3 map/configuration có độ khó tăng dần; báo cáo kích thước state space và action space.
• Trình bày rõ transition, terminal condition và các biến bổ sung để đảm bảo tính Markov.
• Chạy nhiều seed/episode phù hợp, vẽ đường học hoặc thống kê return; trực quan hóa policy cuối cùng trên ít nhất 2 setting.
• So sánh ít nhất 2 thuật toán hoặc 2 biến thể chính sách theo yêu cầu của đề tài; phân tích trường hợp thuật toán thất bại/chậm hội tụ.
• Thử ít nhất 2 cấu hình reward hoặc tham số môi trường và giải thích policy thay đổi.

Sản phẩm tối thiểu
Mã nguồn environment + agent; file cấu hình/map; báo cáo mô tả MDP, thuật toán, thí nghiệm, kết quả; chương trình demo một episode bằng policy đã học.

ĐỀ TÀI 10. Battery Robot

Mục tiêu
Robot phải tới Goal trước khi hết pin; có các trạm sạc.

Mô hình MDP
State: (hàng, cột, mức pin).
Action: Lên, xuống, trái, phải.
Reward gợi ý: Phạt tiêu hao năng lượng; thưởng Goal; hết pin kết thúc episode với penalty.

Các setting bắt buộc
1. Pin giảm 1 mỗi bước.
2. Có trạm sạc đầy pin.
3. Trạm sạc chỉ dùng được 1 lần.
4. Có pin dự phòng nhặt trên map, kích hoạt tự động khi pin về 0 và chỉ dùng một lần.

Yêu cầu riêng
Yêu cầu đánh giá policy khi dung lượng pin thay đổi.

Môi trường / thư viện gợi ý

Custom `gymnasium.Env`; battery là biến rời rạc trong observation/state. Thư viện: gymnasium, numpy, matplotlib.

Thuật toán
Value Iteration và Q-learning. Không sử dụng DQN hoặc function approximation.

Yêu cầu thực nghiệm
• Xây dựng ít nhất 3 map/configuration có độ khó tăng dần; báo cáo kích thước state space và action space.
• Trình bày rõ transition, terminal condition và các biến bổ sung để đảm bảo tính Markov.
• Chạy nhiều seed/episode phù hợp, vẽ đường học hoặc thống kê return; trực quan hóa policy cuối cùng trên ít nhất 2 setting.
• So sánh ít nhất 2 thuật toán hoặc 2 biến thể chính sách theo yêu cầu của đề tài; phân tích trường hợp thuật toán thất bại/chậm hội tụ.
• Thử ít nhất 2 cấu hình reward hoặc tham số môi trường và giải thích policy thay đổi.

Sản phẩm tối thiểu
Mã nguồn environment + agent; file cấu hình/map; báo cáo mô tả MDP, thuật toán, thí nghiệm, kết quả; chương trình demo một episode bằng policy đã học.

ĐỀ TÀI 11. Weighted Terrain Navigation

Mục tiêu
Các ô địa hình có chi phí di chuyển khác nhau như đường, cỏ, bùn, nước.

Mô hình MDP
State: (hàng, cột) và trạng thái vật phẩm nếu có.
Action: Lên, xuống, trái, phải.
Reward gợi ý: Âm theo chi phí của ô đi vào; Goal có reward dương.

Các setting bắt buộc
1. 3 loại terrain.
2. Map lớn hơn và nhiều vùng chi phí cao.
3. Có cầu tắt nhưng chỉ đi qua được một lần rồi sập.
4. Có 'vé địa hình' cho phép đi qua đúng 1 ô nước với chi phí thấp.

Yêu cầu riêng
So sánh shortest path theo số bước với optimal path theo tổng reward.

Môi trường / thư viện gợi ý

Custom `gymnasium.Env` dựa trên GridWorld; terrain-cost lưu trong ma trận chi phí. Thư viện: gymnasium, numpy, matplotlib.

Thuật toán
Value Iteration và Policy Iteration; đối chiếu Q-learning. Không sử dụng DQN hoặc function approximation.

Yêu cầu thực nghiệm
• Xây dựng ít nhất 3 map/configuration có độ khó tăng dần; báo cáo kích thước state space và action space.
• Trình bày rõ transition, terminal condition và các biến bổ sung để đảm bảo tính Markov.
• Chạy nhiều seed/episode phù hợp, vẽ đường học hoặc thống kê return; trực quan hóa policy cuối cùng trên ít nhất 2 setting.
• So sánh ít nhất 2 thuật toán hoặc 2 biến thể chính sách theo yêu cầu của đề tài; phân tích trường hợp thuật toán thất bại/chậm hội tụ.
• Thử ít nhất 2 cấu hình reward hoặc tham số môi trường và giải thích policy thay đổi.

Sản phẩm tối thiểu
Mã nguồn environment + agent; file cấu hình/map; báo cáo mô tả MDP, thuật toán, thí nghiệm, kết quả; chương trình demo một episode bằng policy đã học.

ĐỀ TÀI 12. Delivery Robot một kiện hàng

Mục tiêu
Robot phải tới Pickup, lấy kiện hàng rồi giao tới Destination.

Mô hình MDP
State: (hàng, cột, carrying).
Action: Lên, xuống, trái, phải, Pickup, Drop.
Reward gợi ý: Phạt step; phạt lớn Pickup/Drop sai; thưởng khi giao đúng.

Các setting bắt buộc
1. 1 pickup, 1 destination.
2. Map lớn hơn có obstacle.
3. Có deadline theo số bước.
4. Có ô 'express pass' giảm một nửa penalty thời gian trong 5 bước, chỉ kích hoạt một lần.

Yêu cầu riêng
Bắt buộc xử lý action không hợp lệ rõ ràng trong transition/reward.

Môi trường / thư viện gợi ý

Gymnasium `Taxi-v4` dùng làm mẫu cho pickup/drop-off; khuyến nghị viết custom Env để đổi map, deadline và vật phẩm. Thư viện: gymnasium, numpy, matplotlib.

Thuật toán
Q-learning và SARSA. Không sử dụng DQN hoặc function approximation.

Yêu cầu thực nghiệm
• Xây dựng ít nhất 3 map/configuration có độ khó tăng dần; báo cáo kích thước state space và action space.
• Trình bày rõ transition, terminal condition và các biến bổ sung để đảm bảo tính Markov.
• Chạy nhiều seed/episode phù hợp, vẽ đường học hoặc thống kê return; trực quan hóa policy cuối cùng trên ít nhất 2 setting.
• So sánh ít nhất 2 thuật toán hoặc 2 biến thể chính sách theo yêu cầu của đề tài; phân tích trường hợp thuật toán thất bại/chậm hội tụ.
• Thử ít nhất 2 cấu hình reward hoặc tham số môi trường và giải thích policy thay đổi.

Sản phẩm tối thiểu
Mã nguồn environment + agent; file cấu hình/map; báo cáo mô tả MDP, thuật toán, thí nghiệm, kết quả; chương trình demo một episode bằng policy đã học.

ĐỀ TÀI 13. Multi-Delivery Robot

Mục tiêu
Robot giao nhiều kiện với thứ tự do agent tự quyết định.

Mô hình MDP
State: (hàng, cột, trạng thái từng kiện: chưa lấy/đang mang/đã giao).
Action: Lên, xuống, trái, phải, Pickup, Drop.
Reward gợi ý: Reward theo từng đơn; phạt bước; có thể có đơn ưu tiên.

Các setting bắt buộc
1. 2 kiện độc lập.
2. 3 kiện, chỉ mang được 1 kiện.
3. Một kiện có deadline.
4. Có ô 'tăng tải' cho phép mang 2 kiện cùng lúc nhưng hiệu lực chỉ cho một chuyến.

Yêu cầu riêng
Phải báo cáo kích thước state space và cách mã hóa trạng thái các kiện.

Môi trường / thư viện gợi ý

Custom `gymnasium.Env` phát triển từ logic Taxi; trạng thái kiện hàng mã hóa bằng tuple/bitmask. Thư viện: gymnasium, numpy, matplotlib.

Thuật toán
Q-learning và Monte Carlo trên setting nhỏ. Không sử dụng DQN hoặc function approximation.

Yêu cầu thực nghiệm
• Xây dựng ít nhất 3 map/configuration có độ khó tăng dần; báo cáo kích thước state space và action space.
• Trình bày rõ transition, terminal condition và các biến bổ sung để đảm bảo tính Markov.
• Chạy nhiều seed/episode phù hợp, vẽ đường học hoặc thống kê return; trực quan hóa policy cuối cùng trên ít nhất 2 setting.
• So sánh ít nhất 2 thuật toán hoặc 2 biến thể chính sách theo yêu cầu của đề tài; phân tích trường hợp thuật toán thất bại/chậm hội tụ.
• Thử ít nhất 2 cấu hình reward hoặc tham số môi trường và giải thích policy thay đổi.

Sản phẩm tối thiểu
Mã nguồn environment + agent; file cấu hình/map; báo cáo mô tả MDP, thuật toán, thí nghiệm, kết quả; chương trình demo một episode bằng policy đã học.

ĐỀ TÀI 14. Taxi Grid với đường một chiều

Mục tiêu
Phiên bản Taxi rời rạc có pickup/drop-off và một số đường chỉ đi một hướng.

Mô hình MDP
State: (hàng, cột, vị trí hành khách hoặc in_taxi, đích).
Action: Lên, xuống, trái, phải, Pickup, Drop.
Reward gợi ý: Chuẩn Taxi: step penalty, phạt thao tác sai, thưởng giao đúng.

Các setting bắt buộc
1. Map nhỏ không đường một chiều.
2. Thêm đường một chiều.
3. Một số ngã tư có đèn đóng/mở theo phase.
4. Có 'vé ưu tiên' cho phép bỏ qua đúng 1 lần quy tắc đèn đỏ.

Yêu cầu riêng
Nếu có đèn giao thông, state phải chứa phase thời gian để giữ tính Markov.

Môi trường / thư viện gợi ý

Gymnasium `Taxi-v4` làm baseline trực tiếp; sau đó subclass hoặc viết custom Env để thêm đường một chiều/đèn giao thông. Thư viện: gymnasium, numpy, matplotlib.

Thuật toán
Q-learning và SARSA. Không sử dụng DQN hoặc function approximation.

Yêu cầu thực nghiệm
• Xây dựng ít nhất 3 map/configuration có độ khó tăng dần; báo cáo kích thước state space và action space.
• Trình bày rõ transition, terminal condition và các biến bổ sung để đảm bảo tính Markov.
• Chạy nhiều seed/episode phù hợp, vẽ đường học hoặc thống kê return; trực quan hóa policy cuối cùng trên ít nhất 2 setting.
• So sánh ít nhất 2 thuật toán hoặc 2 biến thể chính sách theo yêu cầu của đề tài; phân tích trường hợp thuật toán thất bại/chậm hội tụ.
• Thử ít nhất 2 cấu hình reward hoặc tham số môi trường và giải thích policy thay đổi.

Sản phẩm tối thiểu
Mã nguồn environment + agent; file cấu hình/map; báo cáo mô tả MDP, thuật toán, thí nghiệm, kết quả; chương trình demo một episode bằng policy đã học.

ĐỀ TÀI 15. Sokoban một thùng – quyền kéo cứu hộ

Mục tiêu
Agent đẩy một thùng vào Goal; bình thường không được kéo thùng.

Mô hình MDP
State: (agent row/col, box row/col, pull_available).
Action: Lên, xuống, trái, phải; Push phát sinh theo chuyển động; Pull chỉ khi đủ điều kiện.
Reward gợi ý: Thưởng khi box vào Goal; phạt bước; phạt khi đẩy box vào dead corner.

Các setting bắt buộc
1. Map 5×5 cơ bản.
2. Map 7×7 có nhiều tường và dead corner.
3. Agent có quyền kéo box đúng 1 lần trong episode.
4. Có ô thưởng 'hook' cho phép khôi phục quyền kéo đúng 1 lần sau khi quyền kéo đã dùng.

Yêu cầu riêng
Bắt buộc tạo ít nhất một map mà quyền kéo có thể cứu box khỏi vùng chết nhưng dùng sai thời điểm sẽ làm policy kém.

Môi trường / thư viện gợi ý

Custom `gymnasium.Env`; không phụ thuộc thư viện Sokoban có sẵn. Map ASCII/NumPy, state gồm agent+box+quyền kéo; pygame dùng để render thao tác đẩy/kéo.

Thuật toán
Q-learning và SARSA; Value Iteration chỉ trên map nhỏ. Không sử dụng DQN hoặc function approximation.

Yêu cầu thực nghiệm
• Xây dựng ít nhất 3 map/configuration có độ khó tăng dần; báo cáo kích thước state space và action space.
• Trình bày rõ transition, terminal condition và các biến bổ sung để đảm bảo tính Markov.
• Chạy nhiều seed/episode phù hợp, vẽ đường học hoặc thống kê return; trực quan hóa policy cuối cùng trên ít nhất 2 setting.
• So sánh ít nhất 2 thuật toán hoặc 2 biến thể chính sách theo yêu cầu của đề tài; phân tích trường hợp thuật toán thất bại/chậm hội tụ.
• Thử ít nhất 2 cấu hình reward hoặc tham số môi trường và giải thích policy thay đổi.

Sản phẩm tối thiểu
Mã nguồn environment + agent; file cấu hình/map; báo cáo mô tả MDP, thuật toán, thí nghiệm, kết quả; chương trình demo một episode bằng policy đã học.

ĐỀ TÀI 16. Sokoban hai thùng – ô cứu hộ

Mục tiêu
Agent phải đưa hai box vào hai target.

Mô hình MDP
State: Vị trí agent, vị trí hai box, trạng thái ô cứu hộ.
Action: Lên, xuống, trái, phải.
Reward gợi ý: Thưởng theo số box đúng target; phạt bước và deadlock.

Các setting bắt buộc
1. Map nhỏ 2 box/2 goal.
2. Map lớn hơn với hành lang hẹp.
3. Có một ô 'rescue' cho phép dịch một box kẹt về vị trí trước đó đúng 1 lần.
4. Ô rescue chỉ kích hoạt nếu agent đi tới ô đó trước khi deadlock xảy ra.

Yêu cầu riêng
Phải chuẩn hóa thứ tự hai box trong state để tránh nhân đôi trạng thái tương đương, hoặc giải thích nếu không làm.

Môi trường / thư viện gợi ý

Custom `gymnasium.Env`; tái sử dụng code Sokoban một thùng, mở rộng state cho hai box và rescue tile. Thư viện: gymnasium, numpy, matplotlib, pygame.

Thuật toán
Q-learning; so sánh với Policy Iteration trên phiên bản thu nhỏ. Không sử dụng DQN hoặc function approximation.

Yêu cầu thực nghiệm
• Xây dựng ít nhất 3 map/configuration có độ khó tăng dần; báo cáo kích thước state space và action space.
• Trình bày rõ transition, terminal condition và các biến bổ sung để đảm bảo tính Markov.
• Chạy nhiều seed/episode phù hợp, vẽ đường học hoặc thống kê return; trực quan hóa policy cuối cùng trên ít nhất 2 setting.
• So sánh ít nhất 2 thuật toán hoặc 2 biến thể chính sách theo yêu cầu của đề tài; phân tích trường hợp thuật toán thất bại/chậm hội tụ.
• Thử ít nhất 2 cấu hình reward hoặc tham số môi trường và giải thích policy thay đổi.

Sản phẩm tối thiểu
Mã nguồn environment + agent; file cấu hình/map; báo cáo mô tả MDP, thuật toán, thí nghiệm, kết quả; chương trình demo một episode bằng policy đã học.

ĐỀ TÀI 17. Ice Sokoban

Mục tiêu
Sokoban có các ô băng; agent hoặc box có thể tiếp tục trượt cho tới khi gặp vật cản.

Mô hình MDP
State: Vị trí agent, box, trạng thái vật phẩm đặc biệt.
Action: Lên, xuống, trái, phải.
Reward gợi ý: Thưởng hoàn thành; phạt bước; deadlock penalty.

Các setting bắt buộc
1. 1 box và vài ô ice.
2. Vùng ice dài, box có thể trượt nhiều ô.
3. Có ice mỏng: box đi qua làm ô đó biến thành ô thường.
4. Có 'spike shoes' dùng 1 lần để dừng agent/box sớm trên ice.

Yêu cầu riêng
Transition của một action có thể đi qua nhiều ô; sinh viên phải mô tả quy tắc cập nhật chính xác.

Môi trường / thư viện gợi ý

Custom `gymnasium.Env`; tái sử dụng Sokoban, thêm loại tile ICE và quy tắc trượt trong `step()`. Thư viện: gymnasium, numpy, pygame.

Thuật toán
Value Iteration trên map nhỏ và Q-learning. Không sử dụng DQN hoặc function approximation.

Yêu cầu thực nghiệm
• Xây dựng ít nhất 3 map/configuration có độ khó tăng dần; báo cáo kích thước state space và action space.
• Trình bày rõ transition, terminal condition và các biến bổ sung để đảm bảo tính Markov.
• Chạy nhiều seed/episode phù hợp, vẽ đường học hoặc thống kê return; trực quan hóa policy cuối cùng trên ít nhất 2 setting.
• So sánh ít nhất 2 thuật toán hoặc 2 biến thể chính sách theo yêu cầu của đề tài; phân tích trường hợp thuật toán thất bại/chậm hội tụ.
• Thử ít nhất 2 cấu hình reward hoặc tham số môi trường và giải thích policy thay đổi.

Sản phẩm tối thiểu
Mã nguồn environment + agent; file cấu hình/map; báo cáo mô tả MDP, thuật toán, thí nghiệm, kết quả; chương trình demo một episode bằng policy đã học.

ĐỀ TÀI 18. Fragile Box Sokoban

Mục tiêu
Một số box dễ vỡ và không được đẩy qua hazard hoặc đẩy quá số lần cho phép.

Mô hình MDP
State: Agent, box, số lần box đã bị đẩy hoặc trạng thái fragile.
Action: Lên, xuống, trái, phải.
Reward gợi ý: Thưởng goal; phạt mỗi push; vỡ box kết thúc episode với penalty.

Các setting bắt buộc
1. Box thường.
2. Fragile box chỉ được push tối đa K lần.
3. Có repair tile đặt lại bộ đếm push về 0 đúng 1 lần.
4. Map nhiều box gồm cả box thường và fragile.

Yêu cầu riêng
Yêu cầu phân tích khi reward shaping theo số lần push làm thay đổi hành vi.

Môi trường / thư viện gợi ý

Custom `gymnasium.Env`; thêm bộ đếm số lần push/trạng thái fragile vào state. Thư viện: gymnasium, numpy, matplotlib, pygame.

Thuật toán
Q-learning và Monte Carlo. Không sử dụng DQN hoặc function approximation.

Yêu cầu thực nghiệm
• Xây dựng ít nhất 3 map/configuration có độ khó tăng dần; báo cáo kích thước state space và action space.
• Trình bày rõ transition, terminal condition và các biến bổ sung để đảm bảo tính Markov.
• Chạy nhiều seed/episode phù hợp, vẽ đường học hoặc thống kê return; trực quan hóa policy cuối cùng trên ít nhất 2 setting.
• So sánh ít nhất 2 thuật toán hoặc 2 biến thể chính sách theo yêu cầu của đề tài; phân tích trường hợp thuật toán thất bại/chậm hội tụ.
• Thử ít nhất 2 cấu hình reward hoặc tham số môi trường và giải thích policy thay đổi.

Sản phẩm tối thiểu
Mã nguồn environment + agent; file cấu hình/map; báo cáo mô tả MDP, thuật toán, thí nghiệm, kết quả; chương trình demo một episode bằng policy đã học.

ĐỀ TÀI 19. Switch–Gate Puzzle

Mục tiêu
Các công tắc thay đổi trạng thái cửa trên map.

Mô hình MDP
State: (hàng, cột, bitmask trạng thái switch/gate).
Action: Lên, xuống, trái, phải, Toggle nếu đứng trên switch.
Reward gợi ý: Thưởng Goal; phạt step; phạt toggle không cần thiết.

Các setting bắt buộc
1. 1 switch–1 gate.
2. 2 switch điều khiển 2 gate.
3. Một switch đảo đồng thời nhiều gate.
4. Có switch đặc biệt chỉ được bật/tắt đúng 1 lần.

Yêu cầu riêng
Phải có map mà trạng thái cửa khiến cùng vị trí agent có giá trị khác nhau rõ rệt.

Môi trường / thư viện gợi ý

MiniGrid DoorKey/Unlock có thể dùng làm tham khảo cho gate/switch; bài nộp nên dùng custom `gymnasium.Env` với bitmask switch. Thư viện: minigrid, gymnasium, numpy.

Thuật toán
Policy Iteration và Q-learning. Không sử dụng DQN hoặc function approximation.

Yêu cầu thực nghiệm
• Xây dựng ít nhất 3 map/configuration có độ khó tăng dần; báo cáo kích thước state space và action space.
• Trình bày rõ transition, terminal condition và các biến bổ sung để đảm bảo tính Markov.
• Chạy nhiều seed/episode phù hợp, vẽ đường học hoặc thống kê return; trực quan hóa policy cuối cùng trên ít nhất 2 setting.
• So sánh ít nhất 2 thuật toán hoặc 2 biến thể chính sách theo yêu cầu của đề tài; phân tích trường hợp thuật toán thất bại/chậm hội tụ.
• Thử ít nhất 2 cấu hình reward hoặc tham số môi trường và giải thích policy thay đổi.

Sản phẩm tối thiểu
Mã nguồn environment + agent; file cấu hình/map; báo cáo mô tả MDP, thuật toán, thí nghiệm, kết quả; chương trình demo một episode bằng policy đã học.

ĐỀ TÀI 20. Pressure Plate Grid

Mục tiêu
Một số cửa chỉ mở khi agent hoặc box đang đứng trên pressure plate.

Mô hình MDP
State: Vị trí agent, box nếu có, trạng thái cửa suy ra từ plate.
Action: Lên, xuống, trái, phải.
Reward gợi ý: Thưởng Goal; phạt step.

Các setting bắt buộc
1. Plate do agent kích hoạt.
2. Thêm 1 box có thể đặt lên plate.
3. Hai plate phải cùng được kích hoạt.
4. Có 'wedge' dùng 1 lần để giữ một cửa mở ngay cả sau khi rời plate.

Yêu cầu riêng
Sinh viên phải chọn state tối thiểu nhưng vẫn Markov; không lưu biến thừa nếu có thể suy ra từ vị trí.

Môi trường / thư viện gợi ý

Custom `gymnasium.Env`; có thể dùng object/tile conventions của MiniGrid để render, nhưng state tabular phải do sinh viên tự mã hóa. Thư viện: gymnasium, minigrid (tùy chọn), numpy.

Thuật toán
Value Iteration và Q-learning. Không sử dụng DQN hoặc function approximation.

Yêu cầu thực nghiệm
• Xây dựng ít nhất 3 map/configuration có độ khó tăng dần; báo cáo kích thước state space và action space.
• Trình bày rõ transition, terminal condition và các biến bổ sung để đảm bảo tính Markov.
• Chạy nhiều seed/episode phù hợp, vẽ đường học hoặc thống kê return; trực quan hóa policy cuối cùng trên ít nhất 2 setting.
• So sánh ít nhất 2 thuật toán hoặc 2 biến thể chính sách theo yêu cầu của đề tài; phân tích trường hợp thuật toán thất bại/chậm hội tụ.
• Thử ít nhất 2 cấu hình reward hoặc tham số môi trường và giải thích policy thay đổi.

Sản phẩm tối thiểu
Mã nguồn environment + agent; file cấu hình/map; báo cáo mô tả MDP, thuật toán, thí nghiệm, kết quả; chương trình demo một episode bằng policy đã học.

ĐỀ TÀI 21. One-Way Arrow Grid

Mục tiêu
Một số ô có mũi tên; khi đứng trên đó chỉ được rời theo một số hướng nhất định.

Mô hình MDP
State: (hàng, cột, trạng thái vé đặc biệt nếu có).
Action: Lên, xuống, trái, phải.
Reward gợi ý: Phạt bước; thưởng Goal; action cấm có penalty hoặc giữ nguyên vị trí.

Các setting bắt buộc
1. Một chiều cố định.
2. Nhiều vùng arrow tạo chu trình.
3. Arrow có thể đảo hướng sau khi qua switch.
4. Có 'override token' cho phép đi ngược mũi tên đúng 1 lần.

Yêu cầu riêng
Phải so sánh hai cách xử lý action không hợp lệ: giữ nguyên + penalty và loại action khỏi tập khả dụng.

Môi trường / thư viện gợi ý

Custom `gymnasium.Env` từ GridWorld; action hợp lệ được kiểm tra theo arrow tile. Thư viện: gymnasium, numpy, matplotlib.

Thuật toán
Value Iteration và Q-learning. Không sử dụng DQN hoặc function approximation.

Yêu cầu thực nghiệm
• Xây dựng ít nhất 3 map/configuration có độ khó tăng dần; báo cáo kích thước state space và action space.
• Trình bày rõ transition, terminal condition và các biến bổ sung để đảm bảo tính Markov.
• Chạy nhiều seed/episode phù hợp, vẽ đường học hoặc thống kê return; trực quan hóa policy cuối cùng trên ít nhất 2 setting.
• So sánh ít nhất 2 thuật toán hoặc 2 biến thể chính sách theo yêu cầu của đề tài; phân tích trường hợp thuật toán thất bại/chậm hội tụ.
• Thử ít nhất 2 cấu hình reward hoặc tham số môi trường và giải thích policy thay đổi.

Sản phẩm tối thiểu
Mã nguồn environment + agent; file cấu hình/map; báo cáo mô tả MDP, thuật toán, thí nghiệm, kết quả; chương trình demo một episode bằng policy đã học.

ĐỀ TÀI 22. Moving Obstacles theo chu kỳ

Mục tiêu
Các obstacle di chuyển theo quỹ đạo định trước sau mỗi bước của agent.

Mô hình MDP
State: (agent row/col, phase của obstacle).
Action: Lên, xuống, trái, phải, Wait.
Reward gợi ý: Thưởng Goal; collision penalty; phạt nhẹ theo bước.

Các setting bắt buộc
1. 1 obstacle chu kỳ 2.
2. Nhiều obstacle chu kỳ khác nhau.
3. Có vùng an toàn cho phép Wait không bị step penalty đúng 1 lần.
4. Có 'freeze tile' đóng băng obstacle trong 2 bước và chỉ kích hoạt một lần.

Yêu cầu riêng
Bắt buộc đưa phase vào state; đánh giá lợi ích của action Wait.

Môi trường / thư viện gợi ý

MiniGrid `DynamicObstacles` dùng làm tham khảo; để phù hợp tabular RL nên viết custom Env với obstacle phase hữu hạn, không dùng ảnh quan sát. Thư viện: minigrid, gymnasium, numpy.

Thuật toán
Q-learning và SARSA. Không sử dụng DQN hoặc function approximation.

Yêu cầu thực nghiệm
• Xây dựng ít nhất 3 map/configuration có độ khó tăng dần; báo cáo kích thước state space và action space.
• Trình bày rõ transition, terminal condition và các biến bổ sung để đảm bảo tính Markov.
• Chạy nhiều seed/episode phù hợp, vẽ đường học hoặc thống kê return; trực quan hóa policy cuối cùng trên ít nhất 2 setting.
• So sánh ít nhất 2 thuật toán hoặc 2 biến thể chính sách theo yêu cầu của đề tài; phân tích trường hợp thuật toán thất bại/chậm hội tụ.
• Thử ít nhất 2 cấu hình reward hoặc tham số môi trường và giải thích policy thay đổi.

Sản phẩm tối thiểu
Mã nguồn environment + agent; file cấu hình/map; báo cáo mô tả MDP, thuật toán, thí nghiệm, kết quả; chương trình demo một episode bằng policy đã học.

ĐỀ TÀI 23. Random Moving Hazard

Mục tiêu
Hazard di chuyển ngẫu nhiên sau mỗi bước; agent phải tới Goal mà tránh va chạm.

Mô hình MDP
State: Vị trí agent và hazard.
Action: Lên, xuống, trái, phải, Wait.
Reward gợi ý: Collision penalty lớn; Goal reward; step penalty.

Các setting bắt buộc
1. 1 hazard random walk.
2. 2 hazard độc lập.
3. Hazard có xác suất đuổi gần agent cao hơn.
4. Có 'smoke bomb' dùng 1 lần làm hazard chuyển sang random walk thuần trong 3 bước.

Yêu cầu riêng
Phải mô tả xác suất transition và kiểm tra tổng xác suất bằng 1.

Môi trường / thư viện gợi ý

MiniGrid `DynamicObstacles` dùng làm tham khảo cho chuyển động vật cản; custom Env tự định nghĩa xác suất random walk. Thư viện: gymnasium, numpy, matplotlib.

Thuật toán
SARSA và Q-learning. Không sử dụng DQN hoặc function approximation.

Yêu cầu thực nghiệm
• Xây dựng ít nhất 3 map/configuration có độ khó tăng dần; báo cáo kích thước state space và action space.
• Trình bày rõ transition, terminal condition và các biến bổ sung để đảm bảo tính Markov.
• Chạy nhiều seed/episode phù hợp, vẽ đường học hoặc thống kê return; trực quan hóa policy cuối cùng trên ít nhất 2 setting.
• So sánh ít nhất 2 thuật toán hoặc 2 biến thể chính sách theo yêu cầu của đề tài; phân tích trường hợp thuật toán thất bại/chậm hội tụ.
• Thử ít nhất 2 cấu hình reward hoặc tham số môi trường và giải thích policy thay đổi.

Sản phẩm tối thiểu
Mã nguồn environment + agent; file cấu hình/map; báo cáo mô tả MDP, thuật toán, thí nghiệm, kết quả; chương trình demo một episode bằng policy đã học.

ĐỀ TÀI 24. Patrolling Guards

Mục tiêu
Các guard đi tuần theo tuyến cố định; agent phải lẻn tới Goal.

Mô hình MDP
State: Vị trí agent, phase của từng guard, trạng thái cloak.
Action: Lên, xuống, trái, phải, Wait.
Reward gợi ý: Bị bắt phạt lớn; Goal thưởng; phạt bước.

Các setting bắt buộc
1. 1 guard tuần tra.
2. 2 guard với chu kỳ khác nhau.
3. Guard có vùng quan sát 1–2 ô.
4. Có cloak dùng đúng 1 lần để vô hiệu vùng quan sát trong 2 bước.

Yêu cầu riêng
Không chỉ tránh ô guard; phải xử lý vùng nhìn và thời điểm sử dụng cloak.

Môi trường / thư viện gợi ý

Custom `gymnasium.Env`; guard path được mô tả bằng chu kỳ hữu hạn, pygame hữu ích cho visualization. Thư viện: gymnasium, numpy, pygame.

Thuật toán
Q-learning và SARSA. Không sử dụng DQN hoặc function approximation.

Yêu cầu thực nghiệm
• Xây dựng ít nhất 3 map/configuration có độ khó tăng dần; báo cáo kích thước state space và action space.
• Trình bày rõ transition, terminal condition và các biến bổ sung để đảm bảo tính Markov.
• Chạy nhiều seed/episode phù hợp, vẽ đường học hoặc thống kê return; trực quan hóa policy cuối cùng trên ít nhất 2 setting.
• So sánh ít nhất 2 thuật toán hoặc 2 biến thể chính sách theo yêu cầu của đề tài; phân tích trường hợp thuật toán thất bại/chậm hội tụ.
• Thử ít nhất 2 cấu hình reward hoặc tham số môi trường và giải thích policy thay đổi.

Sản phẩm tối thiểu
Mã nguồn environment + agent; file cấu hình/map; báo cáo mô tả MDP, thuật toán, thí nghiệm, kết quả; chương trình demo một episode bằng policy đã học.

ĐỀ TÀI 25. Predator–Prey Grid

Mục tiêu
Agent đóng vai predator, mục tiêu là bắt prey có policy cố định hoặc stochastic.

Mô hình MDP
State: Vị trí predator, vị trí prey, trạng thái skill nếu có.
Action: Lên, xuống, trái, phải, Wait.
Reward gợi ý: Thưởng lớn khi bắt; step penalty.

Các setting bắt buộc
1. Prey random walk.
2. Prey ưu tiên chạy xa predator.
3. Map có obstacle.
4. Predator có 'dash' dùng 1 lần để đi 2 ô trong một action nếu đường trống.

Yêu cầu riêng
Policy của prey phải được mô tả cố định để môi trường vẫn là MDP một-agent.

Môi trường / thư viện gợi ý

Custom `gymnasium.Env`; prey dùng policy cố định/random được gọi bên trong `step()`, không dùng PettingZoo để tránh biến thành multi-agent RL. Thư viện: gymnasium, numpy, matplotlib.

Thuật toán
Q-learning và Monte Carlo. Không sử dụng DQN hoặc function approximation.

Yêu cầu thực nghiệm
• Xây dựng ít nhất 3 map/configuration có độ khó tăng dần; báo cáo kích thước state space và action space.
• Trình bày rõ transition, terminal condition và các biến bổ sung để đảm bảo tính Markov.
• Chạy nhiều seed/episode phù hợp, vẽ đường học hoặc thống kê return; trực quan hóa policy cuối cùng trên ít nhất 2 setting.
• So sánh ít nhất 2 thuật toán hoặc 2 biến thể chính sách theo yêu cầu của đề tài; phân tích trường hợp thuật toán thất bại/chậm hội tụ.
• Thử ít nhất 2 cấu hình reward hoặc tham số môi trường và giải thích policy thay đổi.

Sản phẩm tối thiểu
Mã nguồn environment + agent; file cấu hình/map; báo cáo mô tả MDP, thuật toán, thí nghiệm, kết quả; chương trình demo một episode bằng policy đã học.

ĐỀ TÀI 26. Escape from Pursuer

Mục tiêu
Agent phải tới Exit trước khi pursuer bắt kịp.

Mô hình MDP
State: Vị trí agent, pursuer, trạng thái trap.
Action: Lên, xuống, trái, phải, Wait.
Reward gợi ý: Exit reward; bị bắt penalty; step penalty.

Các setting bắt buộc
1. Pursuer random.
2. Pursuer tham lam đi gần agent.
3. Có nhiều Exit với reward khác nhau.
4. Agent có thể đặt một trap dùng 1 lần làm pursuer đứng yên 1 bước.

Yêu cầu riêng
Phải tạo ít nhất một map nơi Exit gần nhưng nguy hiểm và Exit xa nhưng an toàn.

Môi trường / thư viện gợi ý

Custom `gymnasium.Env`; pursuer dùng policy cố định/greedy trong transition. Thư viện: gymnasium, numpy, matplotlib, pygame tùy chọn.

Thuật toán
SARSA và Q-learning. Không sử dụng DQN hoặc function approximation.

Yêu cầu thực nghiệm
• Xây dựng ít nhất 3 map/configuration có độ khó tăng dần; báo cáo kích thước state space và action space.
• Trình bày rõ transition, terminal condition và các biến bổ sung để đảm bảo tính Markov.
• Chạy nhiều seed/episode phù hợp, vẽ đường học hoặc thống kê return; trực quan hóa policy cuối cùng trên ít nhất 2 setting.
• So sánh ít nhất 2 thuật toán hoặc 2 biến thể chính sách theo yêu cầu của đề tài; phân tích trường hợp thuật toán thất bại/chậm hội tụ.
• Thử ít nhất 2 cấu hình reward hoặc tham số môi trường và giải thích policy thay đổi.

Sản phẩm tối thiểu
Mã nguồn environment + agent; file cấu hình/map; báo cáo mô tả MDP, thuật toán, thí nghiệm, kết quả; chương trình demo một episode bằng policy đã học.

ĐỀ TÀI 27. Flood / Lava Spreading Grid

Mục tiêu
Sau mỗi bước, một số ô trở thành nguy hiểm theo quy luật lan truyền.

Mô hình MDP
State: Vị trí agent, phase/thời điểm lan, trạng thái vật phẩm.
Action: Lên, xuống, trái, phải, Wait.
Reward gợi ý: Goal reward; vào ô ngập/lava phạt lớn; step penalty.

Các setting bắt buộc
1. Lava lan theo lịch định trước.
2. Nhiều nguồn lava.
3. Một số tường chặn lan.
4. Có 'barrier' dùng 1 lần để chặn một ô khỏi bị lava trong toàn episode.

Yêu cầu riêng
Nếu quy luật lan phụ thuộc thời gian, bắt buộc đưa time/phase vào state.

Môi trường / thư viện gợi ý

MiniGrid LavaCrossing có thể dùng làm tham khảo tile Lava; phần lan theo thời gian nên cài trong custom `gymnasium.Env`. Thư viện: minigrid, gymnasium, numpy.

Thuật toán
Value Iteration trên setting nhỏ và Q-learning. Không sử dụng DQN hoặc function approximation.

Yêu cầu thực nghiệm
• Xây dựng ít nhất 3 map/configuration có độ khó tăng dần; báo cáo kích thước state space và action space.
• Trình bày rõ transition, terminal condition và các biến bổ sung để đảm bảo tính Markov.
• Chạy nhiều seed/episode phù hợp, vẽ đường học hoặc thống kê return; trực quan hóa policy cuối cùng trên ít nhất 2 setting.
• So sánh ít nhất 2 thuật toán hoặc 2 biến thể chính sách theo yêu cầu của đề tài; phân tích trường hợp thuật toán thất bại/chậm hội tụ.
• Thử ít nhất 2 cấu hình reward hoặc tham số môi trường và giải thích policy thay đổi.

Sản phẩm tối thiểu
Mã nguồn environment + agent; file cấu hình/map; báo cáo mô tả MDP, thuật toán, thí nghiệm, kết quả; chương trình demo một episode bằng policy đã học.

ĐỀ TÀI 28. Fragile Floor / Collapsing Bridge

Mục tiêu
Một số ô chỉ đi qua được số lần hữu hạn; sau đó biến thành hố.

Mô hình MDP
State: Vị trí agent và bitmask/trạng thái các ô fragile.
Action: Lên, xuống, trái, phải.
Reward gợi ý: Goal reward; rơi hố penalty; step penalty.

Các setting bắt buộc
1. Mỗi fragile tile sập sau 1 lần đi qua.
2. Một số tile chịu được 2 lần.
3. Có nhiều route với số fragile khác nhau.
4. Có 'repair kit' dùng đúng 1 lần để khôi phục một tile đã sập.

Yêu cầu riêng
Bắt buộc state hóa lịch sử cần thiết bằng trạng thái tile thay vì lưu toàn bộ trajectory.

Môi trường / thư viện gợi ý

Custom `gymnasium.Env`; trạng thái fragile floor dùng bitmask/counter per tile. Thư viện: gymnasium, numpy, matplotlib.

Thuật toán
Policy Iteration và Q-learning. Không sử dụng DQN hoặc function approximation.

Yêu cầu thực nghiệm
• Xây dựng ít nhất 3 map/configuration có độ khó tăng dần; báo cáo kích thước state space và action space.
• Trình bày rõ transition, terminal condition và các biến bổ sung để đảm bảo tính Markov.
• Chạy nhiều seed/episode phù hợp, vẽ đường học hoặc thống kê return; trực quan hóa policy cuối cùng trên ít nhất 2 setting.
• So sánh ít nhất 2 thuật toán hoặc 2 biến thể chính sách theo yêu cầu của đề tài; phân tích trường hợp thuật toán thất bại/chậm hội tụ.
• Thử ít nhất 2 cấu hình reward hoặc tham số môi trường và giải thích policy thay đổi.

Sản phẩm tối thiểu
Mã nguồn environment + agent; file cấu hình/map; báo cáo mô tả MDP, thuật toán, thí nghiệm, kết quả; chương trình demo một episode bằng policy đã học.

ĐỀ TÀI 29. Timed Gates / Traffic-Light Grid

Mục tiêu
Các cổng mở/đóng theo chu kỳ giống đèn giao thông.

Mô hình MDP
State: (hàng, cột, phase thời gian, pass_available).
Action: Lên, xuống, trái, phải, Wait.
Reward gợi ý: Goal reward; step penalty; đi vào gate đóng bị giữ lại hoặc phạt.

Các setting bắt buộc
1. 1 gate chu kỳ 2.
2. Nhiều gate chu kỳ khác nhau.
3. Có đường vòng không phụ thuộc gate.
4. Có pass dùng đúng 1 lần để đi qua gate đang đóng.

Yêu cầu riêng
Phân tích khi nào action Wait tối ưu và khi nào nên đi đường vòng.

Môi trường / thư viện gợi ý

Custom `gymnasium.Env`; phase của gate/traffic light được cập nhật theo bước và đưa vào state. Thư viện: gymnasium, numpy, matplotlib.

Thuật toán
Value Iteration và SARSA/Q-learning. Không sử dụng DQN hoặc function approximation.

Yêu cầu thực nghiệm
• Xây dựng ít nhất 3 map/configuration có độ khó tăng dần; báo cáo kích thước state space và action space.
• Trình bày rõ transition, terminal condition và các biến bổ sung để đảm bảo tính Markov.
• Chạy nhiều seed/episode phù hợp, vẽ đường học hoặc thống kê return; trực quan hóa policy cuối cùng trên ít nhất 2 setting.
• So sánh ít nhất 2 thuật toán hoặc 2 biến thể chính sách theo yêu cầu của đề tài; phân tích trường hợp thuật toán thất bại/chậm hội tụ.
• Thử ít nhất 2 cấu hình reward hoặc tham số môi trường và giải thích policy thay đổi.

Sản phẩm tối thiểu
Mã nguồn environment + agent; file cấu hình/map; báo cáo mô tả MDP, thuật toán, thí nghiệm, kết quả; chương trình demo một episode bằng policy đã học.

ĐỀ TÀI 30. Rescue Mission nhiều nạn nhân

Mục tiêu
Agent cứu các nạn nhân trên grid rồi đưa về điểm an toàn, với nguồn lực hữu hạn.

Mô hình MDP
State: Vị trí agent, bitmask nạn nhân đã cứu, số medkit còn lại, carrying nếu cần.
Action: Lên, xuống, trái, phải, Rescue/Drop tùy thiết kế.
Reward gợi ý: Reward theo từng nạn nhân; bonus khi cứu đủ; phạt bước và nguy hiểm.

Các setting bắt buộc
1. 2 nạn nhân, không giới hạn nguồn lực.
2. 3–4 nạn nhân, chỉ mang 1 người/lần.
3. Một số nạn nhân cần medkit; medkit hữu hạn.
4. Có ô supply cho phép bổ sung đúng 1 medkit nhưng phải đi đường vòng.

Yêu cầu riêng
Phải có ít nhất hai policy hợp lý cạnh tranh nhau: cứu gần trước hoặc ưu tiên nạn nhân giá trị/cấp bách cao.

Môi trường / thư viện gợi ý

Custom `gymnasium.Env`; map, victim mask, medkit và carrying state đều rời rạc. Thư viện: gymnasium, numpy, matplotlib; pygame cho demo.

Thuật toán
Monte Carlo và Q-learning; có thể thêm Policy Iteration trên map nhỏ. Không sử dụng DQN hoặc function approximation.

Yêu cầu thực nghiệm
• Xây dựng ít nhất 3 map/configuration có độ khó tăng dần; báo cáo kích thước state space và action space.
• Trình bày rõ transition, terminal condition và các biến bổ sung để đảm bảo tính Markov.
• Chạy nhiều seed/episode phù hợp, vẽ đường học hoặc thống kê return; trực quan hóa policy cuối cùng trên ít nhất 2 setting.
• So sánh ít nhất 2 thuật toán hoặc 2 biến thể chính sách theo yêu cầu của đề tài; phân tích trường hợp thuật toán thất bại/chậm hội tụ.
• Thử ít nhất 2 cấu hình reward hoặc tham số môi trường và giải thích policy thay đổi.

Sản phẩm tối thiểu
Notebook hoặc project gồm: environment + agent, file cấu hình/map, báo cáo mô tả MDP, thuật toán, thí nghiệm, kết quả, chương trình demo một episode bằng policy đã học.